CAutomation Project Engineering Contract

Authoritative platform-level engineering contract

Status: Authoritative project-level platform contract for CAutomation. This document is intentionally platform-level. Module-specific requirements, screens, entities, endpoints, data fields, schemas, workflows, and acceptance details belong in module-level contracts. When a project-level decision and a module-level detail conflict, the project-level decision controls unless the project-level contract is formally changed.

Review rule applied in this iteration: every section answers the question, "If this were the only information available to the AI about this topic, could it make the correct architectural decision?" Sections have been enriched only where needed to remove platform-level ambiguity.

1. Document Purpose and Engineering Boundary

This contract defines how the CAutomation platform must be engineered at project level. It gives the AI enough shared engineering authority to generate a complete web application without inventing architecture, stack, security, API, data, logging, validation, or deployment rules.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Use this document for shared engineering decisions that every module must inherit.
- Do not use it to define module-specific endpoints, schemas, fields, screens, workflows, or reports.
- If module technical detail is missing, the AI must report a module-contract gap rather than placing that detail in the project contract.

2. Engineering Principles

These principles are mandatory across the generated application.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Contract-first: approved project-level and module-level contracts are the source of truth.
- Validation-first: invalid, unsupported, ambiguous, or incomplete inputs must fail before downstream generation or acceptance.
- Traceability-first: generated artifacts, execution reports, logs, validation results, and approvals must reference their source contracts and execution identity.
- Separation of concerns: UI, API, application service, domain, repository, persistence, AI execution, and reporting responsibilities must remain distinct.
- Modular but integrated: modules must be independently understandable but operate inside one shared platform shell.
- Explicit over implicit: hidden defaults, inferred permissions, unrecorded approvals, and silent fallback behavior are not acceptable.
- Whole-file generation: AI-generated source changes must be produced as complete files, not partial patches, unless a later project-level rule explicitly permits another mode.

3. Platform Architecture

CAutomation must be generated as an end-to-end web application with a clear frontend, backend API, application service layer, domain model, repositories, database, AI execution layer, validation/reporting layer, and export layer.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Layer	Mandatory responsibility	Must not contain
Frontend web application	React TypeScript UI for navigation, administration, contract management, execution review, reports, approvals, and exports.	Database access or business-rule enforcement that belongs on the backend.
Backend API	FastAPI REST API exposing authenticated platform and module operations.	Frontend rendering logic or direct AI provider coupling in controllers.
Application services	Use-case orchestration, authorization checks, lifecycle transitions, validation coordination, and transaction boundaries.	Raw SQL scattered through controllers or UI concerns.
Domain model	Shared concepts: Client, Project, Module, Contract, Pipeline, Task, Execution, Report, Deliverable, Role, Permission, Approval.	Framework-specific persistence or transport details.
Repositories	Persistence access and query encapsulation.	Business orchestration or authorization policy decisions.
Database	PostgreSQL persistence for platform state, lifecycle, ownership, traceability, reports, and deliverables.	Business logic implemented only as database side effects.
AI execution layer	Provider-independent execution, staging, manifest, prompt, raw response, and generated artifact handling.	Direct writes to accepted project state without validation/apply control.
Validation/reporting	Structured checks and machine-readable reports for inputs, outputs, artifacts, and executions.	Unstructured messages as the only evidence.
Export layer	Packaging approved deliverables with reports and provenance.	Bypassing approval or validation state.

4. Technology Stack

The stack must be explicit so generated code remains consistent across modules.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Area	Decision	Rationale / boundary
Frontend	React with TypeScript.	Single web UI with typed components and API clients.
Backend	Python FastAPI.	Clear REST APIs, validation models, and service layering.
Database	PostgreSQL.	Relational persistence for platform state and traceability.
Data validation	Pydantic-style request, response, and internal validation models.	Do not rely on untyped dictionaries for core API contracts.
Testing	Unit, functional, validation, task, pipeline, AI Engine, and CAutomation-level runners where applicable.	Runners must produce structured reports for GUI consumption.
Documents	PDF as primary supported source contract format at this stage.	DOCX support may exist but must not drive equal current engineering effort.
AI provider	Provider abstraction with provider-specific integrations behind a stable contract.	Application logic must not depend directly on one provider implementation.

5. Project Structure Standards

Generated artifacts must preserve a predictable structure so the platform, tests, reports, and future GUI tooling can find assets consistently.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- The root project name is CAutomation. Legacy names must not be generated.

- Project-level contracts belong under project-level contract locations; module-level contracts belong under each module.
- Each pipeline and task must keep its executable code, tests, fixtures, reports, and documentation discoverable by level.
- Generated output must separate source contracts, normalized inputs, staging workspaces, reports, manifests, logs, and accepted deliverables.
- No module may write directly into another module ownership area except through approved platform APIs or generated integration contracts.

6. Backend Standards

Backend generation must follow layered service architecture and explicit validation.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- API routes accept and return typed request/response models.
- Routes delegate business behavior to services; they do not implement complex domain workflows directly.
- Services enforce authorization, lifecycle validity, cross-module rules, and transaction boundaries.
- Repositories encapsulate persistence access and return domain or data-transfer objects, not UI-specific structures.
- Validation failures must return structured errors with actionable messages and machine-readable codes where practical.
- Long-running or pipeline-like operations must expose execution state rather than blocking silently.
- File and artifact handling must preserve provenance, safe paths, and controlled lifecycle state.

7. Frontend Standards

Frontend generation must produce a coherent platform UI rather than isolated module screens.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- The UI must provide a shared application shell with navigation for clients, projects, modules, contracts, executions, reports, deliverables, approvals, and exports.
- Screens must show current context: selected client, selected project, selected module where applicable, status, owner, blockers, and next action.
- Frontend code must use typed API clients or typed request/response shapes; avoid ad hoc JSON assumptions.
- Validation and execution errors must be visible to users in business language, with enough detail to act.
- Reusable components should be used for tables, status badges, evidence panels, approval actions, report viewers, and artifact lists.
- The frontend must not implement security by hiding UI only; backend authorization remains authoritative.

8. Database Standards

Database design must support isolation, lifecycle state, traceability, and auditability across all modules.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Core records must include stable identity, ownership context, lifecycle/status fields, timestamps, and actor references where applicable.
- Client-owned and project-owned data must carry enough keys to enforce visibility and authorization.
- Contract, execution, report, deliverable, and approval records must preserve source/version relationships.
- Use migrations for schema changes; generated code must not assume manual database edits.

- Prefer soft deletion, archive, or supersession for business-significant records. Hard deletion must be explicitly justified.
- Database constraints should support integrity, but business workflows must still be expressed in services.

9. API Standards

The API must be consistent enough for the AI to generate frontend and backend behavior without inventing patterns.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- REST resources should reflect platform concepts: clients, projects, modules, contracts, pipelines, tasks, executions, reports, deliverables, approvals, users, roles, permissions.
- Endpoints must use consistent plural nouns, stable identifiers, and project/client context where required.
- Requests and responses must be typed and documented through the backend framework where possible.
- Errors must be structured and should distinguish validation failure, authorization failure, missing resource, conflict/lifecycle violation, and unexpected server failure.
- Operations that change approval, execution, export, or lifecycle state must be explicit commands rather than hidden side effects of read operations.
- Pagination, filtering, and sorting should be available for list views that can grow over time.

10. Security and Authorization Principles

Security is platform-level and cannot be weakened by module contracts.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Authentication is required for non-public platform operations.
- Authorization must be enforced on the backend using roles, permissions, and ownership context.
- Client isolation is mandatory. Users must not read or mutate other clients data without explicit cross-client platform authority.
- Project membership limits project visibility and actions.
- Approval and administrative operations require specific permissions and must be auditable.
- Secrets, provider keys, and deployment configuration must not be committed into source contracts or generated source files.
- File paths, uploaded documents, generated artifacts, and exports must be validated to prevent unsafe access outside approved workspace boundaries.

11. Quality and Test Standards

Quality standards must make generated output inspectable and suitable for future GUI reporting.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Unit tests verify small services, validators, repositories, and components.
- Functional tests verify platform workflows across APIs and important frontend/backend seams.
- Validation tests verify contract normalization, input readiness, artifact manifests, reports, and acceptance blockers.
- Task, pipeline, AI Engine, and CAutomation-level runners must use names that identify execution level clearly.
- Test reports must be structured and machine-readable enough for later web GUI consumption.
- A passing implementation is not enough if reports, manifests, traceability, or validation evidence are missing.
- Regression tests should protect platform concepts and cross-module rules from accidental drift.

12. Logging, Monitoring, and Reports

The platform must make operational and generation behavior understandable without relying on hidden console output.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Use structured logging for important platform, security, validation, execution, and export events.
- Logs should include execution identifiers, project context, module context, task identity, and error category when applicable.
- Reports must be generated for normalization, validation, execution, apply/export, and final run summaries where relevant.
- Reports must distinguish success, warning, failure, skipped, and blocked states.
- Business users should inspect reports through the platform UI; engineers should be able to inspect raw structured report files.
- Do not use print statements as the primary operational reporting mechanism.

13. AI Generation Standards

AI-generated behavior must remain bounded by approved contracts and controlled workspaces.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- The AI must use approved contracts and normalized inputs as authority; it must not read unrelated live repository files to invent requirements.
- Generation must write to a staging workspace first, with prompt, raw response, manifest, generated files, and generation report preserved.
- Generated artifacts must be whole files with explicit paths and manifest entries.
- Schema mismatch, missing required fields, unsafe paths, or unsupported operations must block apply/acceptance.
- Provider-specific details must remain behind a provider abstraction.
- If a required decision is missing from contracts, the generated report must identify the gap instead of silently inventing the answer.
- AI outputs must preserve CAutomation naming and must not reintroduce legacy project names.

14. Cross-Module Engineering Rules

Modules must integrate through shared platform rules rather than duplicating platform infrastructure.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Modules inherit authentication, authorization, client/project context, contract lifecycle, execution traceability, validation reporting, approval, export, and logging rules.
- Modules may define their own domain entities and workflows only inside module contracts.
- A module must not define a parallel user model, permission model, approval model, execution model, or report model.
- Shared UI components and backend services should be reused for common platform concepts.
- Cross-module references must be explicit and validated; hidden coupling through file paths or duplicated identifiers is not acceptable.

15. Deployment and Environment Principles

Deployment expectations must be explicit enough to avoid environment-specific invention.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- The application must support separate configuration for local development, test, and production-like environments.
- Secrets and provider credentials must come from environment/configuration mechanisms, not hard-coded files.
- Database migrations and initial setup must be repeatable.
- Generated reports, staging workspaces, and exports must use configurable safe locations.
- The platform should be container-friendly, but exact production infrastructure choices are outside this contract unless defined later.
- Deployment must preserve the ability to inspect logs, reports, generated artifacts, and validation evidence.

16. Engineering Acceptance Criteria

The generated platform is technically acceptable only when platform-level engineering rules are implemented consistently across modules.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Frontend, backend, database, validation, reporting, AI execution, and export behavior work together as one CAutomation platform.
- Project-level contracts and module-level contracts remain synchronized in generated outputs.
- Security, authorization, client isolation, and project membership are enforced server-side.
- Every business-significant generated artifact is traceable to approved source contracts and execution evidence.
- Failed validation blocks acceptance and produces actionable structured reports.
- Tests and runners produce structured reports suitable for later GUI consumption.
- No generated file uses legacy project names or module-specific behavior in project-level locations.

17. Engineering Glossary

The AI must use these engineering terms consistently.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Term	Engineering definition
Application service	Backend layer that orchestrates business use cases, authorization, validation, and transactions.
Repository	Backend layer that encapsulates persistence access and queries.
Execution identity	Stable identifier connecting a pipeline/task run to inputs, outputs, logs, reports, and artifacts.
Manifest	Structured inventory of generated artifacts, paths, checksums, and metadata.
Staging workspace	Controlled location where AI-generated output is written before validation/apply/acceptance.
Validation report	Machine-readable evidence describing checks, results, failures, warnings, and blockers.
Apply	Controlled promotion of validated generated artifacts into an accepted target location.
Structured error	Error response with category, message, and details that software and users can act on.
Provider abstraction	Stable interface hiding provider-specific AI implementation details.
Scope drift	Any generated change outside the approved iteration boundary.